

Rapport de Travaux Pratiques

Clavier Souple 4×4_ Système de Serrure à Code

Affichage sur LCD avec ATmega2560

Environnement : Atmel Studio

Professeur : Mr BARRO

Réalisé par : Payié Mariane ELOLA
Sephora Maurane BAYALA
Mamadou Latif SALIFOU HAMANI

Année Académique : 2025-2026

SOMMAIRE

1.	INTRODUCTION.....	3
2.	Matériel et environnement de développement.....	3
2.1.	Composants matériels	3
2.2.	Affectation des ports	3
2.3.	Environnement logiciel	4
3.	Montage.....	4
4.	Logigramme	6
5.	Architecture Générale du programme.....	7
5.1.	Module LCD.....	7
5.2.	Module Clavier.....	7
5.3.	Module LEDs	7
6.	Fonctionnement de base du système	8
6.1.	Ecran d'accueil	8
6.2.	Saisie du code.....	8
6.3.	Vérification et retour visuel	8
7.	Nouvelles Fonctionnalités.....	8
7.1.	Limitation du nombre de tentatives	8
7.2.	Blocage temporel après des échecs répétés	9
7.3.	Effacement du dernier caractère.....	10
7.4.	Annulation et retour à l'accueil.....	11
7.5.	Récapitulatif des fonctionnalités ajoutées.....	12
8.	Constantes et Paramétrage	12
9.	Conclusion.....	12

1. INTRODUCTION

Ce rapport présente la conception et la réalisation d'un système de serrure électronique à code numérique, basé sur le microcontrôleur ATmega2560. Le projet s'inscrit dans le cadre des travaux pratiques du module Pratique des microprocesseurs.

Dans sa version de base, le système permet à un utilisateur de saisir un code confidentiel via un clavier matriciel souple 4×4 et d'obtenir un retour visuel immédiat : une LED verte s'allume en cas de code correct, tandis qu'une LED rouge signale une erreur. L'afficheur LCD 16×2 guide l'utilisateur tout au long du processus de saisie.

Au-delà de cette fonctionnalité de base, nous avons choisi d'enrichir le système avec plusieurs fonctionnalités avancées : limitation du nombre de tentatives, blocage temporel en cas d'échecs répétés, touche d'effacement et touche d'annulation. Ces ajouts, détaillés dans la section dédiée, visent à rapprocher le prototype d'un produit réel et sécurisé.

2. Matériel et environnement de développement

2.1. Composants matériels

Composant	Référence	Rôle
Microcontrôleur	ATmega2560	Unité centrale de traitement
Afficheur LCD	LCD 16×2 (HD44780)	Interface utilisateur visuelle
Clavier	Matriciel souple 4×4	Saisie du code utilisateur
LED verte	PB0	Signal d'accès autorisé
LED rouge	PB1	Signal d'accès refusé / blocage
Potentiomètre	10 kΩ	Réglage du contraste du LCD
Alimentation	5 V DC	Alimentation du système
Fils de connexion	Jumper wires	Connexions entre composants
Breadboard	830 points	Support de câblage sans soudure

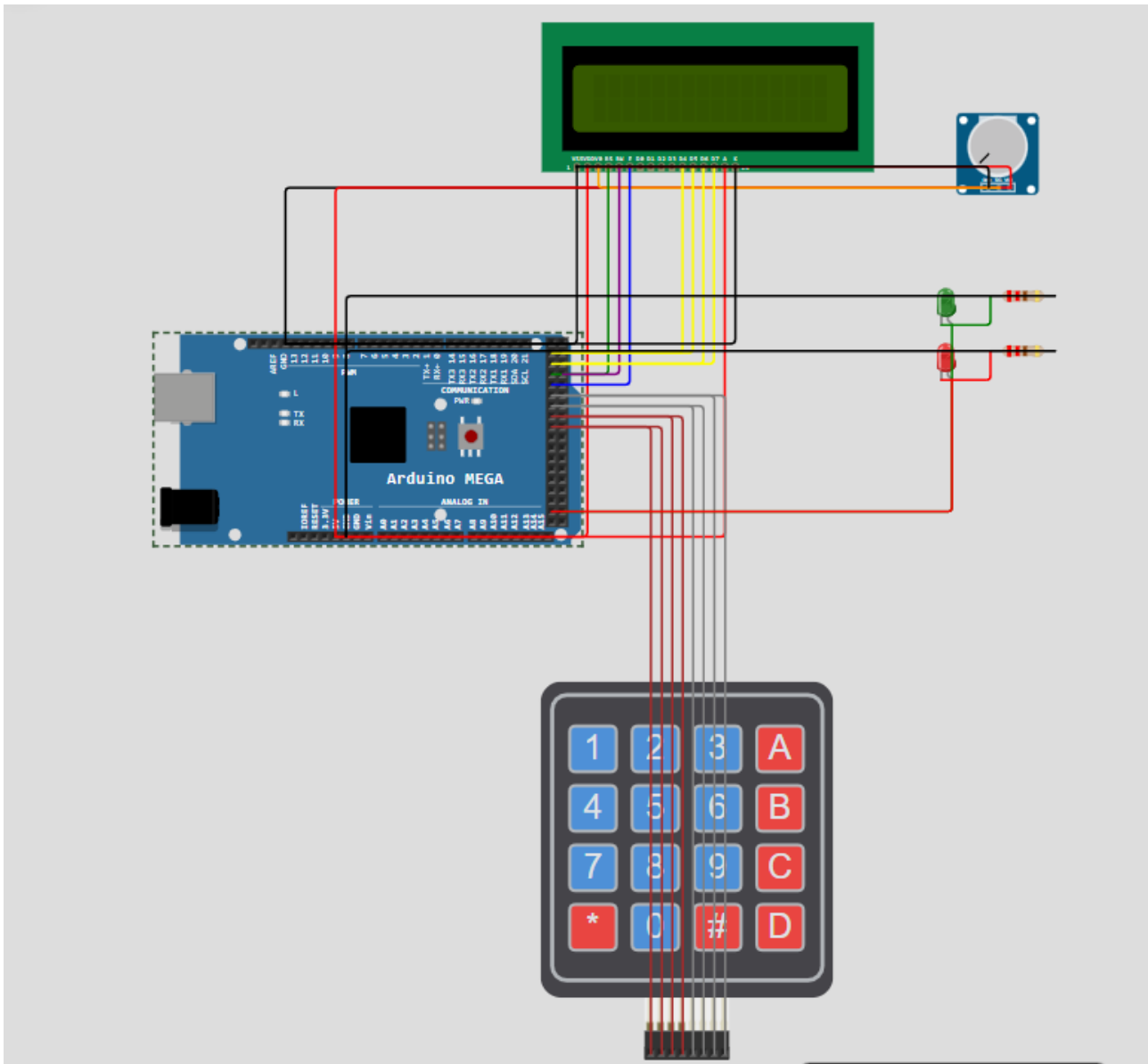
2.2. Affectation des ports

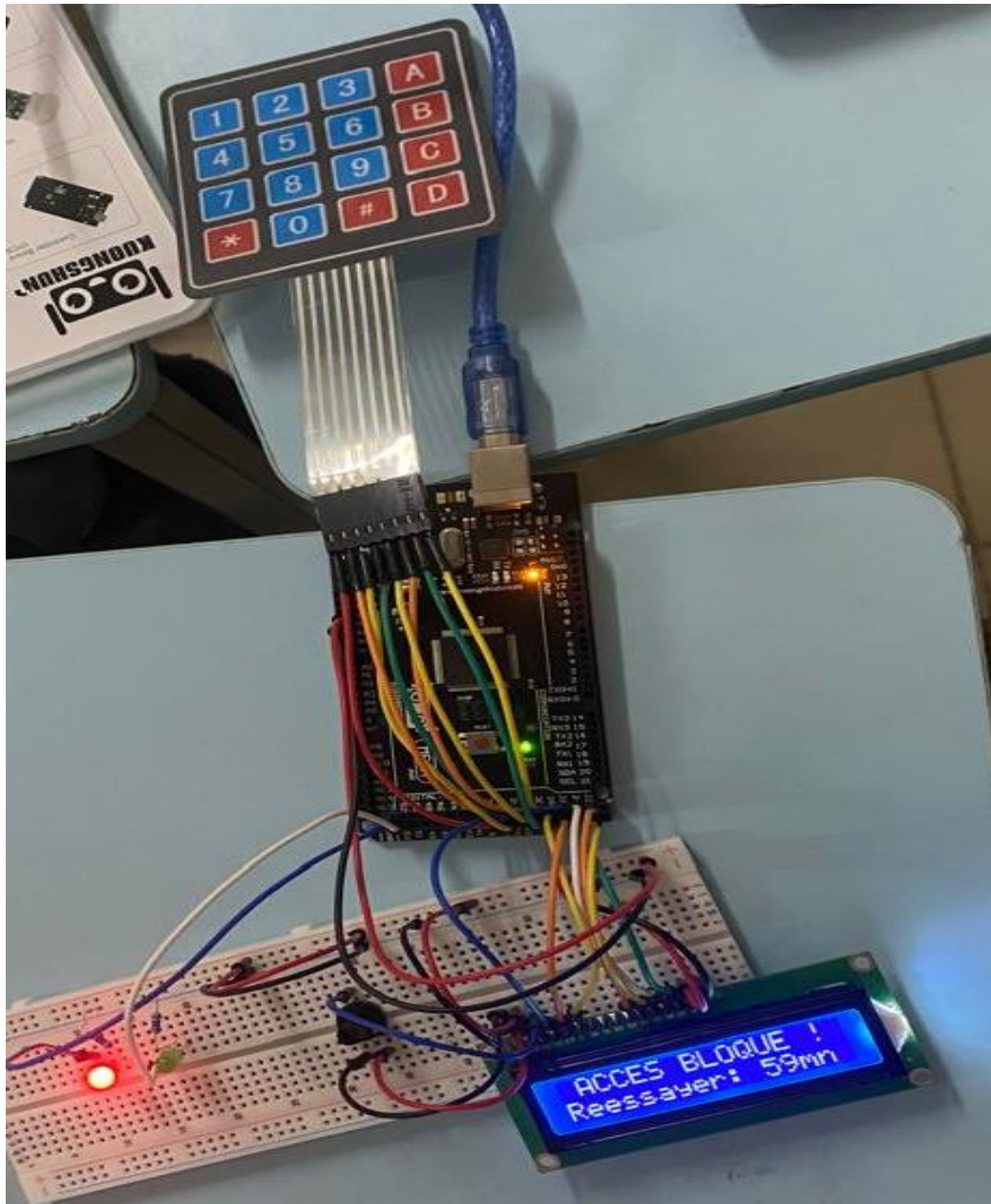
Port	Broche(s)	Fonction
PORT A	PA0–PA6	Bus de données et contrôle LCD (D4–D7, RS, RW, EN)
PORT B	PB0, PB1	LED verte et LED rouge
PORT C	PC0–PC3 (sorties)	Lignes du clavier matriciel
PORT C	PC4–PC7 (entrées)	Colonnes du clavier matriciel
VCC / GND	Broches alimentation	Alimentation 5 V pour tous les périphériques

2.3. Environnement logiciel

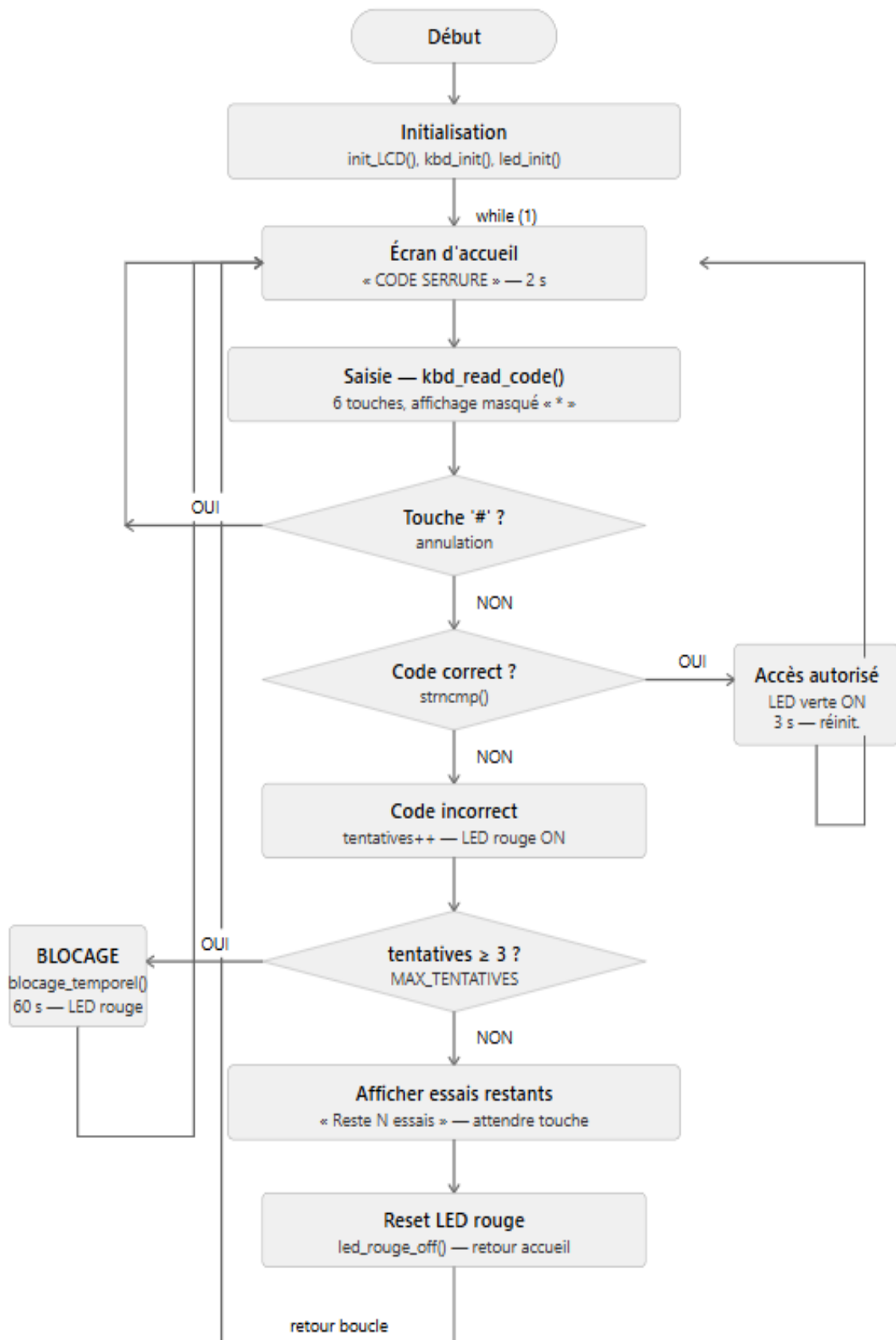
- **Langage** : C standard (C99), compilé avec avr-gcc
- **IDE** : Microchip Studio (AVR Studio)
- **Bibliothèques** : <avr/io.h>, <util/delay.h>, <string.h>, <stdio.h>
- **Fréquence CPU** : $F_{CPU} = 16 \text{ MHz}$

3. Montage





4. Logigramme



5. Architecture Générale du programme

Le programme est structuré de manière modulaire, chaque composant matériel disposant de son propre ensemble de fonctions dédiées. Cette approche facilite la lisibilité, la maintenance et l'évolution du code. On distingue quatre modules principaux.

5.1. Module LCD

Le pilotage de l'afficheur LCD suit le protocole de communication 4 bits du contrôleur HD44780. La communication en mode 4 bits réduit le nombre de broches utilisées (4 bits de données au lieu de 8) au prix d'un protocole légèrement plus complexe : chaque octet est transmis en deux séquences successives (quartet haut puis quartet bas).

- **init_LCD()** : Initialise l'afficheur selon la séquence de démarrage HD44780 (mode 4 bits, 2 lignes, curseur invisible).
- **Write_Cmd()** / **Write_Cmd1()** : Envioient des commandes de contrôle à l'afficheur (effacement, positionnement du curseur, etc.).
- **Write_Data()** : Envoie un caractère ASCII à afficher, en maintenant RS=1.
- **Write_LCD()** : Affiche une chaîne de caractères complète en appelant Write_Data() pour chaque octet.
- **pos_xy(), ligne_1(), ligne_2()** : Positionnent le curseur à une colonne et une ligne précises.
- **cls_LCD()** : Efface l'écran et repositionne le curseur en (1,1).

5.2. Module Clavier

Le clavier matriciel souple 4×4 est composé de 4 lignes et 4 colonnes, soit 16 touches. La technique de balayage (scanning) consiste à activer une ligne à la fois (niveau bas) et à lire l'état des colonnes pour détecter quelle touche est enfoncée.

- **kbd_init()** : Configure PC0–PC3 en sorties (lignes) et PC4–PC7 en entrées avec résistances pull-up internes activées.
- **kbd_get_key()** : Parcourt les 4 lignes, met chacune à 0 tour à tour, lit les colonnes inversées (actif bas) et retourne le caractère correspondant dans la table keymap[4][4]. Inclut un anti-rebond logiciel (attente de 20 ms et relâchement de touche).

5.3. Module LEDs

Deux LEDs sont utilisées comme indicateurs visuels de l'état du système. Leur pilotage est simplifié par quatre fonctions élémentaires.

- **led_init()** : Configure PB0 et PB1 en sorties, LEDs éteintes à l'initialisation.
- **led_verte_on/off()** : Allume ou éteint la LED verte (PB0) pour signaler un accès autorisé.
- **led_rouge_on/off()** : Allume ou éteint la LED rouge (PB1) pour signaler un refus ou un blocage.
-

6. Fonctionnement de base du système

Le comportement fondamental du système, tel que défini dans le cahier des charges initial, repose sur trois phases successives : l'affichage de l'écran d'accueil, la saisie du code confidentiel, et la vérification avec retour visuel.

6.1. Ecran d'accueil

À chaque cycle, le LCD affiche un message d'accueil sur deux lignes pendant 2 secondes (`DELAI_ACCUEIL_MS = 2000 ms`). Ce délai donne à l'utilisateur le temps de se préparer avant la saisie. Le message affiché est :

```
Ligne 1 :  [ CODE SERRURE ]  
Ligne 2 :  [Entrez le code:]
```

6.2. Saisie du code

La fonction `kbd_read_code()` gère l'interaction complète avec l'utilisateur. Elle attend que l'utilisateur appuie sur exactement 6 touches (`CODE_LEN = 6`). Pour préserver la confidentialité, chaque caractère saisi est affiché sous la forme d'un astérisque '*' sur la ligne 2 du LCD, tandis que le vrai caractère est stocké en mémoire dans le tableau `saisie[]`.

La ligne 1 affiche en permanence le nombre d'essais restants, permettant à l'utilisateur de savoir combien de tentatives il lui reste avant le blocage.

6.3. Vérification et retour visuel

Une fois les 6 caractères saisis, la fonction `strcmp()` compare le code entré avec la constante `SECRET_CODE` définie en tête de programme ("123456" par défaut). Deux scénarios sont possibles :

- **Code correct** : La LED verte s'allume, le LCD affiche « ACCES AUTORISE » et « Bienvenue ! » pendant 3 secondes (`DELAI_SUCCES_MS`), puis le système retourne à l'accueil.
- **Code incorrect** : La LED rouge s'allume, le LCD affiche « Code incorrect » ainsi que le nombre d'essais restants pendant 500 ms (`DELAI_ECHEC_MS`). Le compteur de tentatives est incrémenté.

7. Nouvelles Fonctionnalités

Les fonctionnalités décrites dans cette section ont été ajoutées afin d'améliorer la sécurité et la fluidité du système. Elles s'inspirent du comportement des systèmes de contrôle d'accès réels.

7.1. Limitation du nombre de tentatives

Un système de contrôle d'accès sans limitation de tentatives est vulnérable aux attaques. La limitation à 3 tentatives consécutives (`MAX_TENTATIVES = 3`) est une contre-mesure classique utilisée dans les systèmes réels (distributeurs automatiques de billets, mot de passe des smartphones).

Un compteur d'échecs consécutifs (`uint8_t tentatives`) est déclaré dans la boucle principale. Il s'incrémente à chaque code incorrect et se remet à zéro dès qu'un code correct est entré.

```
uint8_t tentatives = 0;
```

```

// Après un code incorrect :
tentatives++;
// Affichage du nombre d'essais restants :
uint8_t restants = MAX_TENTATIVES - tentatives;
afficher_essais(restants);
// Après un code correct :
tentatives = 0; // remise à zéro

```

La fonction `afficher_essais()` adapte son message au nombre restant : elle distingue le cas singulier (1 essai) du cas pluriel (2 essais) pour soigner l'interface utilisateur.

```

void afficher_essais(uint8_t restants) //Affiche le nombre d'essais
restants sur la ligne 1
{
    char buf[17]; //buffer temporaire pour construire le texte
    pos_xy(1, 1); //positionnement début ligne 1
    if (restants == 1) //si 1 seul essai restant
        snprintf(buf, sizeof(buf), "Saisie[1 essai] ");
    else //plusieurs essais restants
        snprintf(buf, sizeof(buf), "Saisie[%u essais]", (unsigned)restants);
    Write_LCD(buf); //affichage final sur LCD
}

```

7.2. Blocage temporel après des échecs répétés

Ce mécanisme, appelé « time-out » ou « cool-down », est présent dans presque tous les systèmes d'authentification modernes.

La fonction `blocage_temporel()` est appelée lorsque `tentatives >= MAX_TENTATIVES`. Elle exécute un compte à rebours minute par minute, affichant en temps réel le temps restant avant le déblocage. La LED rouge reste allumée pendant toute la durée du blocage.

```

void blocage_temporel(void) //pour un compte à rebours
{
    uint16_t minutes = DELAI_BLOCAGE_S / 60u; //Conversion secondes en
minutes
    led_rouge_on();
    for (uint16_t m = minutes; m > 0; m--)
    {
        char buf[17];
        cls_LCD(); //nettoyage écran
        pos_xy(1, 1); //ligne 1 : message principal
        Write_LCD(" ACCES BLOQUE ! ");
        pos_xy(1, 2); //ligne 2 : temps restant
    }
}

```

```

    snprintf(buf, sizeof(buf), "Reessayer: %2umn  ", (unsigned)m);
    Write_LCD(buf);
    /* Attendre 60 secondes (60 × 1000 ms) */
    for (uint8_t s = 0; s < 60; s++)
        _delay_ms(1000);
}
led_rouge_off();//à fin du blocage
}

```

Points techniques notables :

- **Double boucle for** : La boucle externe compte les minutes, la boucle interne compte 60 secondes de 1000 ms chacune. Cette décomposition évite les dépassements de valeur (`_delay_ms()` a une limite sur AVR).
- **Remise à zéro du compteur** : Après le blocage, tentatives est remis à 0 afin que l'utilisateur reparte avec un crédit complet de 3 essais.
- **LED rouge maintenue** : Elle reste allumée visuellement pendant tout le blocage, fournissant un retour matériel supplémentaire.

7.3. Effacement du dernier caractère

Sans possibilité de correction, une erreur de frappe oblige l'utilisateur à valider un code incorrect, consommant ainsi inutilement une tentative. La touche '*' permet de supprimer le dernier caractère saisi, rendant l'interface plus confortable et réduisant les erreurs de manipulation — surtout lorsque le LCD affiche des '*' masquant les vraies valeurs.

À l'intérieur de la boucle de saisie `kbd_read_code()`, la détection de '*' décrémente l'index de saisie `idx` et efface visuellement l'astérisque correspondant sur le LCD.

```

if (k == '*')//TOUCHE '*' pour supprimer le dernier caractère
{
    if (idx > 0)
    {
        idx--; //revenir d'un caractère
        effacer_char_lcd(idx + 1); //efface le '*' affiché
    }
    continue;
}

```

La fonction `effacer_char_lcd()` positionne le curseur sur la colonne correspondante à la ligne 2 et y écrit un espace, effaçant ainsi visuellement le caractère précédent.

```

void effacer_char_lcd(uint8_t col)//Efface le caractère sur la ligne 2 à la colonne indiquée

```

```
{
    pos_xy(col, 2); //aller à la bonne colonne
    Write_Data(' '); //affiche un espace à la place
}
```

Mécanisme de protection : la condition `if (idx > 0)` empêche une décrémentation en dessous de zéro, évitant tout comportement indéfini si l'utilisateur appuie sur '*' alors qu'aucun caractère n'a encore été saisi.

7.4. Annulation et retour à l'accueil

Cette touche offre à l'utilisateur la possibilité d'interrompre à tout moment une saisie en cours pour revenir à l'écran d'accueil, sans que cela ne compte comme une tentative ratée. Cela est particulièrement utile si l'utilisateur réalise en cours de saisie qu'il va faire une erreur, ou s'il a simplement déclenché accidentellement le clavier.

La détection de '#' dans la boucle de saisie provoque un affichage bref de confirmation sur le LCD (« Annulation / Retour accueil »), puis la fonction retourne la valeur 0. La boucle principale interprète ce retour comme un signal de réinitialisation et repart directement à l'accueil via `continue`, sans incrémenter le compteur de tentatives.

```
if (k == '#') //TOUCHE '#' pour annulation complète
{
    cls_LCD();
    pos_xy(1, 1);
    Write_LCD("  Annulation  ");
    pos_xy(1, 2);
    Write_LCD("  Retour accueil");
    _delay_ms(1500);
    return 0;    //signale l'annulation au main()
}
```

Dans le `main()`

```
uint8_t complet = kbd_read_code(saisie, restants); //lecture du code
if (!complet) //si l'utilisateur appuie sur '#' on recommence directement
sans pénalité
    continue;
```

Le retour sans pénalité est rendu possible par la vérification de la valeur de retour de `kbd_read_code()`. Lorsque `complet == 0` (annulation), le bloc de vérification du code est entièrement court-circuité grâce à l'instruction `continue`, préservant intégralement le compteur tentatives.

7.5. Récapitulatif des fonctionnalités ajoutées

Fonctionnalité	Déclencheur	Comportement LCD	Impact sécurité
Limitation tentatives	3 codes incorrects	Essais restants ligne 1	Résistance à la force brute
Blocage temporel	tentatives ≥ 3	ACCES BLOQUE + compte à rebours	Ralentissement des attaques
Effacement (*)	Touche * enfoncée	Astérisque supprimé ligne 2	Réduction erreurs utilisateur
Annulation (#)	Touche # enfoncée	Message annulation 1.5 s	Aucune pénalité de tentative

8. Constantes et Paramétrage

L'ensemble des paramètres critiques du système est centralisé sous forme de constantes en tête du fichier source. Cette approche facilite la modification et l'adaptation du système sans avoir à parcourir l'intégralité du code.

Constante	Valeur	Signification
CODE_LEN	6	Longueur du mot de passe (6 chiffres)
MAX_TENTATIVES	3	Nombre maximal de tentatives avant blocage
SECRET_CODE	"123456"	Code confidentiel stocké en mémoire flash
DELAI_ACCUEIL_MS	2000	Durée d'affichage de l'écran d'accueil (ms)
DELAI_SUCCES_MS	3000	Durée d'affichage du message de succès (ms)
DELAI_ECHEC_MS	500	Pause après affichage d'un code incorrect (ms)
DELAI_BLOCAGE_S	60	Durée totale du blocage temporel (secondes)
F_CPU	16 000 000	Fréquence d'horloge du microcontrôleur (Hz)

9. Conclusion

Ce projet de travaux pratiques nous a permis de mettre en œuvre un système embarqué complet, alliant pilotage de périphériques (LCD, clavier matriciel, LEDs) et logique applicative de sécurité sur microcontrôleur ATmega2560.

Partant d'une base fonctionnelle simple — saisie d'un code et retour LED — nous avons enrichi le système avec quatre fonctionnalités avancées inspirées des systèmes réels : la limitation à trois tentatives, le blocage temporel d'une minute, la correction par la touche '*' et l'annulation sans pénalité par la touche '#'. Chacune de ces fonctionnalités répond à un besoin concret de sécurité ou d'ergonomie.

Sur le plan technique, ce projet a renforcé notre maîtrise de la programmation en C pour systèmes embarqués : gestion des ports E/S, communication en mode 4 bits avec un afficheur LCD, balayage matriciel avec anti-rebond, et structuration modulaire du code.